

Spotiflac Dummy Files

Referencia Tecnica Completa

Documento de contexto para trabajo en local con LLM. Incluye el problema, arquitectura interna de Spotiflac, esquemas de bases de datos, descripciones de archivos generados, y flujo de trabajo completo.

App: SpotiFLAC Mobile v4.8.5
Dispositivo: HiBy R1 + Android
Fecha: 2026-08-16

1. El Problema	4
1.1 Configuración relevante de Spotiflac	4
2. Arquitectura Interna de Spotiflac	4
2.1 Componentes de almacenamiento	4
2.2 El campo match_key (campo crítico)	4
2.3 Cruce de datos: Playlist vs Historial	5
3. Esquemas de Bases de Datos	5
3.1 history.db - Tabla "history"	5
3.2 history.db - Tabla "history_path_keys"	6
3.3 local_library.db - Tabla "library"	6
3.4 Rutas SAF - Formato de rutas de Android	7
4. Archivos Generados	7
4.1 spotiflac_dummy_generator.py	7
Uso del script	7
Formatos soportados y tamaño de dummy	7
4.2 history.db (modificada)	8
4.3 local_library.db (vacía)	8
4.4 spotiflac_dummy_889.zip (Enfoque A - generado previamente)	8
5. Flujo de Trabajo	9
5.1 Flujo principal (Enfoque A: archivos dummy)	9
5.2 Flujo alternativo (Enfoque B: modificar bases de datos)	9
6. Notas Críticas para el LLM Local	9
6.1 El match_key es el campo crítico	9
6.2 Los path_keys son variantes de ruta	9
6.3 audio_metadata_scan_version	10
6.4 SAF y las rutas en Android	10
6.5 El backup JSON no es la fuente de verdad	10
6.6 El usuario tiene root	10
6.7 Si el Enfoque B no funciona	10

6.8 Playlist "En Híby" - Estructura 10

1. El Problema

El usuario descarga musica desde la app **SpotiFLAC** (v4.8.5) en Android y la transfiere a un reproductor portatil **HiBy R1**. El telefono tiene espacio limitado, asi que borra las canciones despues de transferirlas. Al borrar los archivos, Spotiflac pierde la referencia de que esas canciones ya fueron descargadas, y al buscar canciones para descargar nueva musica, **no muestra la etiqueta de duplicado**, lo que lleva a descargar la misma cancion varias veces, desperdiciando tiempo y ancho de banda.

El objetivo es hacer creer a Spotiflac que las canciones siguen en el telefono para que siga mostrando la etiqueta de duplicado al buscar, sin ocupar espacio real con los archivos de audio completos. Se exploraron dos enfoques: (A) archivos dummy en disco y (B) modificacion directa de las bases de datos internas de la app.

1.1 Configuracion relevante de Spotiflac

Setting	Valor	Descripcion
downloadDirectory	/storage/emulated/0/Music/SpotyFlac	Carpeta de descarga
storageMode	saf	Storage Access Framework
filenameFormat	{title} - {artist}	Formato de nombre de archivo
localLibraryEnabled	true	Biblioteca local activa
localLibraryShowDuplicates	true	Mostrar duplicados activo
localLibraryAutoScan	off	Escaneo automatico desactivado
deduplicateDownloads	true	Deduplicacion de descargas activa
saveDownloadHistory	true	Guardar historial activo

2. Arquitectura Interna de Spotiflac

Se analizo un backup JSON exportado desde la app y se extrajeron las bases de datos SQLite del telefono (ruta: /data/data/com.zarz.spotiflac/app_flutter/). Spotiflac usa tres sistemas para rastrear canciones, cada uno con un proposito especifico en la deteccion de duplicados y la gestion de la biblioteca musical del usuario.

2.1 Componentes de almacenamiento

Componente	Tipo	Contenido	Filas
history.db	SQLite	Historial de descargas con match_key	756
local_library.db	SQLite	Archivos escaneados en disco	889
library_collections.db	SQLite	Vacio (sin tablas)	0
app_state.db	SQLite	Vacio (sin tablas)	0
backup.json	JSON	Playlists, configuracion, loved tracks	-

Las playlists y la configuracion de la app **NO** se almacenan en SQLite. Las bases de datos library_collections.db y app_state.db estan completamente vacias (sin tablas). Toda la informacion de playlists y configuraciones se guarda exclusivamente en el archivo JSON de backup que el usuario exporta manualmente.

2.2 El campo match_key (campo critico)

El campo match_key es el campo mas importante para la deteccion de duplicados. Su formato es siempre {track_name_norm}|{artist_name_norm} donde ambos valores estan en minusculas y normalizados. Spotiflac usa este

campo para comparar canciones al buscar y determinar si ya fueron descargadas. Este campo esta presente tanto en `history.db` como en `local_library.db` con el mismo formato, lo que sugiere que la app puede consultar cualquiera de las dos bases de datos (o ambas) para marcar duplicados.

Ejemplos de `match_key`: "there's nothing holdin' me back|shawn mendes" "baby said|maneskin"
"dig|unlike pluto" "come alive|pendulum" "generation love|ar/co, punctual, newera"

2.3 Cruce de datos: Playlist vs Historial

La playlist "En Hiby" contiene 889 canciones que el usuario ya transfirió al HiBy R1. Al cruzar esta playlist con el historial de descargas (756 entradas), se encontró que 609 canciones ya estaban en el historial, pero **280 canciones faltaban**. Estas 280 son las que no se marcaban como duplicado al buscar, lo cual es el núcleo del problema que se busca resolver. Los formatos de audio reales de la biblioteca del usuario son predominantemente FLAC (92%), con pequeñas cantidades de OPUS, AAC y M4A para canciones donde ese formato no estaba disponible.

Metrica	Valor
Canciones en playlist "En Hiby"	889
Entradas en historial de descargas	756
Canciones ya en historial (cruce por ISRC/match_key)	609
Canciones FALTANTES en historial	280
Formato dominante	FLAC (92%)
Otros formatos	OPUS (10), AAC (26), M4A (19)
Canciones marcadas como "loved"	2,095

3. Esquemas de Bases de Datos

3.1 history.db - Tabla "history"

Esta tabla almacena cada canción descargada a través de la app. Es el registro permanente de descargas. Cada fila incluye metadatos completos de la canción, la ruta del archivo (en formato SAF), y los campos normalizados usados para búsqueda y detección de duplicados. El campo `id` es una clave primaria compuesta por el Spotify ID, un timestamp de milisegundos y un número secuencial.

Columna	Tipo	Nulable	Descripción
<code>id</code>	TEXT	NO	Clave primaria: {spotify_id}-{timestamp}-{seq}
<code>track_name</code>	TEXT	NO	Nombre de la canción
<code>artist_name</code>	TEXT	NO	Artista principal
<code>album_name</code>	TEXT	NO	Nombre del álbum
<code>album_artist</code>	TEXT	SI	Artista del álbum
<code>cover_url</code>	TEXT	SI	URL de la caratula (Spotify CDN)
<code>file_path</code>	TEXT	NO	Ruta SAF completa al archivo
<code>storage_mode</code>	TEXT	SI	Modo de almacenamiento (saf/file)
<code>download_tree_uri</code>	TEXT	SI	URI del árbol SAF
<code>saf_relative_dir</code>	TEXT	SI	Subdirectorío relativo SAF
<code>saf_file_name</code>	TEXT	SI	Nombre del archivo en SAF

Columna	Tipo	Nulable	Descripcion
saf_repaired	INTEGER	SI	Si la ruta fue reparada (0/1)
service	TEXT	NO	Servicio de descarga (tidal-web, etc.)
downloaded_at	TEXT	NO	Timestamp de descarga (ISO 8601)
isrc	TEXT	SI	Codigo ISRC de la cancion
spotify_id	TEXT	SI	ID de Spotify de la cancion
quality	TEXT	SI	Calidad (ej: 24-bit/44.1kHz)
bit_depth	INTEGER	SI	Profundidad de bits
sample_rate	INTEGER	SI	Frecuencia de muestreo
bitrate	INTEGER	SI	Tasa de bits
format	TEXT	SI	Formato (flac, opus, aac, m4a)
match_key	TEXT	SI	CLAVE: {nombre_lower} {artista_lower}
isrc_norm	TEXT	SI	ISRC en minusculas
search_text	TEXT	SI	Texto concatenado para busqueda

3.2 history.db - Tabla "history_path_keys"

Esta tabla mapea cada cancion del historial a multiples variantes de su ruta de archivo. Android SAF usa URIs que pueden estar percent-encoded o no, en mayusculas o minusculas, y con diferentes formatos de separador. Cada cancion tiene aproximadamente 12 variantes de ruta almacenadas para asegurar que la app pueda encontrar el archivo independientemente del formato de URI que devuelva el sistema de archivos en cada momento.

Columna	Tipo	Descripcion
item_id	TEXT (PK)	Referencia al id de la tabla history
path_key	TEXT (PK)	Variante de la ruta del archivo

3.3 local_library.db - Tabla "library"

Esta tabla almacena los archivos de audio encontrados al escanear la carpeta de descarga. A diferencia del historial, esta tabla se pobla dinamicamente al escanear el sistema de archivos, no por descargas realizadas a traves de la app. El campo audio_metadata_scan_version indica si se leyeron los metadatos del archivo (0 = escaneo rapido de nombre solo, 1 = escaneo completo con lectura de tags).

Columna	Tipo	Descripcion
id	TEXT (PK)	ID unico: lib_{sha1_hash}
track_name	TEXT (NO)	Nombre de la cancion (desde metadatos o nombre de archivo)
artist_name	TEXT (NO)	Artista (desde metadatos o nombre de archivo)
album_name	TEXT	Album
album_artist	TEXT	Artista del album
file_path	TEXT (NO)	Ruta SAF del archivo escaneado
cover_path	TEXT	Ruta a la caratula local
scanned_at	TEXT (NO)	Timestamp del escaneo (ISO 8601)
file_mod_time	INTEGER	Fecha de modificacion del archivo (epoch ms)
isrc	TEXT	ISRC leido de los metadatos del archivo
duration	INTEGER	Duracion en segundos (NULL si no se leyo metadata)

Columna	Tipo	Descripcion
format	TEXT	Formato detectado (flac, opus, m4a, etc.)
bit_depth	INTEGER	Profundidad de bits (NULL si no se leyo metadata)
sample_rate	INTEGER	Frecuencia de muestreo (NULL si no se leyo metadata)
bitrate	INTEGER	Tasa de bits (NULL si no se leyo metadata)
match_key	TEXT	CLAVE: {nombre_lower} {artista_lower} (igual formato que history)
audio_metadata_scan_version	INTEGER (def 0)	0=rapido, 1=completo con lectura de tags

Nota: La tabla library_path_keys tiene la misma estructura que history_path_keys (item_id + path_key como PK compuesta). Cada item de la biblioteca tambien tiene aproximadamente 12 variantes de ruta.

3.4 Rutas SAF - Formato de rutas de Android

Spotiflac usa Storage Access Framework (SAF) para acceder a los archivos. Las rutas son URIs de contenido de Android con codificacion percent-encoding. Cada archivo tiene multiples variantes de ruta almacenadas en las tablas path_keys. A continuacion se muestran las tres variantes principales que se encontraron para un mismo archivo:

```

Variante 1 (percent-encoded, mayusculas): content://com.android.externalstorage.documents/tree/primary%3AMusic%2FSpotyFlac/document/primary%3AMusic%2FSpotyFlac%2F...
Variante 2 (percent-encoded, minusculas): content://com.android.externalstorage.documents/tree/primary%3amusic%2fspotyflac/document/primary%3amusic%2fspotyflac%2f...
Variante 3 (sin encoding, con caracteres reales): content://com.android.externalstorage.documents/tree/primary:Music/SpotyFlac/document/primary:Music/SpotyFlac/...

```

4. Archivos Generados

4.1 spotiflac_dummy_generator.py

Propiedad	Valor
Ubicacion	/home/z/my-project/download/spotiflac_dummy_generator.py
Tipo	Script Python reutilizable
Dependencia	mutagen (pip install mutagen)
Proposito	Genera ZIP con archivos dummy a partir de musica real en PC

Este script es la herramienta principal para el usuario. Se ejecuta en la PC apuntando a la carpeta donde el usuario tiene su musica real (la misma que transfiere al HiBy R1). Lee cada archivo de audio, extrae sus metadatos reales (titulo, artista, album, genero, ISRC) usando la libreria mutagen, y genera un archivo dummy tiny con el mismo nombre y extension. Los FLAC dummy contienen un header FLAC valido con metadatos VORBIS_COMMENT reales incrustados. Los demas formatos generan stubs minimos con headers validos. Todo se empaqueta en un ZIP que el usuario descomprime en la carpeta SpotyFlac del telefono.

Uso del script

```

# Instalar dependencia pip install mutagen # Modo original (mantiene nombre real del archivo)
python spotiflac_dummy_generator.py -i "C:\ruta\a\musica" # Modo spotiflac (genera nombre estilo "{Titulo} - {Artista}.ext")
python spotiflac_dummy_generator.py -i "C:\Music" --naming spotiflac # Especificar salida
python spotiflac_dummy_generator.py -i "D:\Music\HiByR1" -o dummies.zip

```

Formatos soportados y tamaño de dummy

Formato	Contenido del dummy	Tamaño aprox.
FLAC	Header valido + STREAMINFO + VORBIS_COMMENT con metadata real	200-500 bytes
OPUS	Header OGG + pagina OpusHead	80 bytes
OGG	Header OGG + pagina OpusHead	80 bytes
M4A/MP4	Boxes ftyp + moov vacios	40 bytes
AAC	Header ADTS + frame dummy	23 bytes
MP3	Tag ID3v2 con titulo/artista + frame MP3	450 bytes
WAV	Header RIFF/WAVE + fmt + data vacio	58 bytes
AIFF	Header FORM/AIFF + COMM + SSND	80 bytes
APE	Header APETAGEX	40 bytes
WavPack	Header wvpk	30 bytes
WMA	Header ASF con GUID	30 bytes

4.2 history.db (modificada)

Propiedad	Valor
Ubicacion	/home/z/my-project/download/history.db
Tipo	Base de datos SQLite
Tamaño	8.3 MB
Proposito	Reemplazar history.db del telefono para agregar 280 canciones faltantes

Se copio la base de datos original (756 entradas) y se agregaron 280 nuevas filas correspondientes a las canciones de la playlist "En Híby" que no estaban en el historial. Para cada cancion faltante se genero una entrada completa con: un ID unico (lib_hash + timestamp + hash aleatorio), el match_key correctamente formateado, isrc_norm en minusculas, una ruta SAF apuntando a SpotyFlac (el archivo no existe fisicamente), y el search_text concatenado para busqueda. Tambien se insertaron las entradas correspondientes en history_path_keys con las variantes de ruta (lowercase, con/sin percent-encoding). El total final es de 1036 entradas con 1036 match_keys unicos verificados.

Incertidumbre: No se sabe si Spotiflac usa history.db SOLO para detectar duplicados, o si requiere tambien local_library.db. Si solo usa history, este enfoque funciona perfecto. Si requiere local_library, sera necesario mantener los archivos dummy en disco o insertar entradas directamente en local_library.db.

4.3 local_library.db (vacuada)

Propiedad	Valor
Ubicacion	/home/z/my-project/download/local_library.db
Tipo	Base de datos SQLite
Tamaño	8.2 MB
Proposito	Reemplazar local_library.db para limpiar la biblioteca de archivos dummy

Se copio la base de datos original y se ejecutaron DELETE FROM en las tablas library (de 889 a 0 filas) y library_path_keys. La estructura de la base de datos se mantiene intacta (tablas, indices, esquemas), solo se eliminaron los datos. El tamaño de 8.2 MB se debe al espacio pre-asignado por SQLite. Al restaurar esta base de datos en el telefono, la pestana "Biblioteca" de Spotiflac deberia quedar vacia, sin mostrar los archivos dummy con iconos de error.

4.4 spotiflac_dummy_889.zip (Enfoque A - generado previamente)

Propiedad	Valor
Ubicacion	/home/z/my-project/download/spotiflac_dummy_889.zip
Tipo	Archivo ZIP comprimido
Tamano comprimido	199 KB
Tamano descomprimido	~3.5 MB (889 archivos)
Contenido	834 FLAC + 26 AAC + 19 M4A + 10 OPUS
Metadatos	Basados en backup JSON (no en archivos reales)

Este ZIP fue generado a partir del backup JSON, no de los archivos reales de musica. Los metadatos se extrajeron de las entradas del historial y de la playlist. Los 286 tracks que no se pudieron cruzar con el historial se asignaron formato FLAC por defecto. Si el formato real era distinto, Spotiflac podria no detectarlos como duplicado. **El script** `spotiflac_dummy_generator.py` **es la version mejorada** que lee metadatos reales de los archivos y deberia usarse en lugar de este ZIP para resultados mas precisos.

5. Flujo de Trabajo

5.1 Flujo principal (Enfoque A: archivos dummy)

1. Descargar cancion en Spotiflac (Android)
2. Transferir la cancion al HiBy R1 (via USB/cable)
3. Borrar la cancion del telefono (por espacio)
4. Ejecutar `spotiflac_dummy_generator.py` en PC: `python spotiflac_dummy_generator.py -i "C:\ruta\musica\en\PC"`
5. Pasar el ZIP generado al telefono
6. Descomprimir el ZIP en `/storage/emulated/0/Music/SpotyFlac/`
7. Spotiflac detecta los dummies al escanear la biblioteca local
8. Al buscar canciones, muestra la etiqueta de duplicado

Nota: Los dummies aparecen en la pestana Biblioteca con icono rojo de error. Este es un efecto secundario conocido del Enfoque A.

5.2 Flujo alternativo (Enfoque B: modificar bases de datos)

1. Descargar cancion en Spotiflac (Android)
2. Transferir la cancion al HiBy R1 (via USB/cable)
3. Borrar la cancion del telefono (por espacio)
4. Reemplazar `history.db` y `local_library.db` en el telefono via MiXplorer: `/data/data/com.zarz.spotiflac/app_flutter/`
5. Forzar cierre de Spotiflac desde ajustes de Android
6. Volver a abrir Spotiflac
7. Los duplicados se detectan desde el historial, sin archivos dummy

Ventaja: La biblioteca queda limpia (sin iconos de error) Riesgo: Si Spotiflac requiere `local_library.db` para duplicados, no funciona.

6. Notas Criticas para el LLM Local

6.1 El `match_key` es el campo critico

El campo `match_key` es el campo mas importante para la deteccion de duplicados. Su formato es `{titulo_normalizado}|{artista_normalizado}` donde ambos valores estan en minusculas. Este campo DEBE coincidir exactamente con lo que Spotiflac genera al buscar una cancion. Si el normalizado no es correcto (por ejemplo, caracteres especiales, acentos, o separadores diferentes), la deteccion de duplicados fallara para esa cancion.

6.2 Los `path_keys` son variantes de ruta

Cada cancion tiene aproximadamente 12 entradas en las tablas `path_keys` correspondientes. Estas son variantes de la ruta del archivo con diferentes codificaciones URL (SAF percent-encoded, lowercase, sin encoding). Al insertar nuevas entradas en el historial o la biblioteca, es necesario generar estas variantes para que la app pueda encontrar el archivo

independientemente del formato que devuelva el sistema.

6.3 audio_metadata_scan_version

El campo `audio_metadata_scan_version` en `local_library` indica el nivel de escaneo: 0 significa escaneo rapido que solo lee el nombre del archivo (sin metadatos), mientras que 1 indica un escaneo completo que lee los tags de metadatos del archivo de audio. Los archivos dummy generados con metadatos FLAC reales deberian pasar a version 1 si Spotiflac los reescanea.

6.4 SAF y las rutas en Android

Spotiflac usa Storage Access Framework (SAF) para acceder a los archivos en Android moderno. Las rutas son URIs de contenido, no rutas de filesystem tradicionales. El formato base es: `content://com.android.externalstorage.documents/tree/{encoded_tree}/document/{encoded_path}` La misma ruta puede aparecer en multiples variantes (percent-encoded, lowercase, sin encoding), y todas deben estar en las tablas `path_keys` para una deteccion confiable.

6.5 El backup JSON no es la fuente de verdad

El archivo JSON de backup que exporta Spotiflac contiene playlists y configuracion, pero las bases de datos SQLite son las que Spotiflac consulta en tiempo real para la deteccion de duplicados y la biblioteca local. Modificar el JSON y restaurarlo actualizara playlists y configuracion, pero no afectara el historial de descargas ni la biblioteca local. Para esos, se deben modificar directamente las bases de datos SQLite.

6.6 El usuario tiene root

El usuario tiene acceso root en su dispositivo Android y usa MiXplorer para navegar y modificar archivos en `/data/data/com.zarz.spotiflac/`. Esto permite modificar directamente las bases de datos SQLite de la app, algo que no seria posible sin root. Al reemplazar archivos de bases de datos, es importante mantener los permisos originales (MiXplorer suele hacerlo automaticamente al sobrescribir) y forzar el cierre completo de la app antes de reabrirla.

6.7 Si el Enfoque B no funciona

Si Spotiflac requiere `local_library.db` para detectar duplicados (y no solo `history.db`), una alternativa seria insertar entradas directamente en `local_library.db` sin archivos fisicos. Las entradas tendrian datos completos (duration, quality, bitrate, etc.) pero el `file_path` apuntaria a archivos que no existen fisicamente. Si Spotiflac solo verifica la base de datos al buscar (y no valida la existencia del archivo en ese momento), los duplicados se detectarían sin necesidad de archivos dummy. Si la app valida la existencia del archivo, esta alternativa tampoco funcionaria y el Enfoque A (archivos dummy reales) seria la unica solucion viable.

6.8 Playlist "En Hiby" - Estructura

La playlist "En Hiby" tiene 889 tracks almacenados en el JSON de backup bajo `data.collections.playlists[1].tracks`. Cada track tiene la estructura:

```
{ "key": "isrc:RUAJF2601781", // o "tidal-web:xxx", o "amazon:xxx" "track": { "id":  
  "lib_3c737ae7f8e51cdf...", // hash unico "name": "Beasts", "artistName": "Edgars Bukovskis, Buko,  
  Megazvers", "albumName": "Beasts", "albumArtist": "Edgars Bukovskis", "isrc": "RUAJF2601781",  
  "duration": 178, "releaseDate": "2026-07-19", "source": "local", "coverUrl":  
  "/data/user/0/.../cover_xxx.jpg" }, "addedAt": "2026-08-04T18:40:24.122178" }
```

El campo `key` usa el prefijo para indicar la fuente de la canción. El 99.4% de los tracks usan `isrc:` (883 de 889), seguido por `tidal-web:` (5) y `amazon:` (1). El `id` de cada track empieza con `lib_` seguido de un hash SHA1, lo que indica que fueron agregados a la playlist desde la biblioteca local. El campo `source:` `"local"` confirma esto.